

BITÁCORA DEL LABORATORIO

Raspberry Pi 4 + ESP32 — Todo lo que hemos aprendido y hecho

Raspberry Pi 4 8GB • ESP32-D0WD-V3 • MobaXterm • MicroPython • Abril 2026



SECCIÓN 1: EL HARDWARE

Qué tenemos y cómo funciona cada pieza

Raspberry Pi 4 Model B — 8GB RAM

¿Qué es?

Una mini computadora completa del tamaño de una tarjeta de crédito. Corre Linux real (Raspbian/Debian), tiene puertos USB, Ethernet Gigabit, WiFi, Bluetooth y 4 núcleos ARM a 1.8GHz. No es una 'computadora de juguete' — es un servidor Linux completo.

Especificaciones clave

CPU: ARM Cortex-A72 quad-core 1.8GHz
RAM: 8GB LPDDR4
Red: Gigabit Ethernet + WiFi 802.11ac
Bluetooth: 5.0
Almacenamiento: MicroSD 32GB
Consumo: ~5W (casi nada)

PiSwitch + CanaKit

El Pi no tiene botón de encendido propio. El PiSwitch es un interruptor que va entre el cargador USB-C y el Pi. El CanaKit provee exactamente 5V/3A que el Pi necesita. Usar cargadores genéricos puede dañarlo.

Por qué sirve para ciberseguridad

Es Linux real en hardware real. Puedes montar servidores, honeypots, sniffers, firewalls y laboratorios de ataque/defensa. Consume tan poco que puedes dejarlo encendido 24/7 esperando ataques reales.

ESP32-D0WD-V3 — El Microcontrolador

¿Qué es la ESP32?

Un microcontrolador (no una computadora completa) con WiFi y Bluetooth integrado. Modelo detectado: ESP32-D0WD-V3 rev 3.1. MAC: 44:1d:64:f9:69:f4. Dual Core 240MHz, 520KB RAM. Se programa con MicroPython o Arduino IDE.

Diferencia con el Pi

El Pi es una computadora (corre Linux, tiene sistema operativo). La ESP32 es un microcontrolador (no tiene SO, corre un solo programa). Son complementarios: el Pi procesa y analiza, la ESP32 sensa y genera tráfico.

Comandos clave detectados durante la sesión:

```
# Detectar ESP32 en el Pi
ls /dev/ttyUSB*                          # Muestra /dev/ttyUSB0 si está conectada
python3 -m esptool --port /dev/ttyUSB0 chip_id   # Lee modelo, MAC, frecuencia
# Resultado: ESP32-D0WD-V3 rev3.1 | WiFi+BT | 240MHz | MAC: 44:1d:64:f9:69:f4
```



SECCIÓN 2: CONFIGURACIÓN INICIAL

Cómo dejamos todo listo desde cero

El camino hasta SSH — Problemas reales y soluciones

⚠ SD original no arrancaba

→ SD dañada — vendedor reemplazó

⚠ Sin luz verde al encender

→ hdmi_force_hotplug=1 en config.txt de bootfs

⚠ TV sin señal HDMI

→ Usar HDMI0 (puerto más cercano al USB-C)

⚠ Error de escritura en Imager

→ Asignar letra de unidad en Administración de Discos

⚠ Access denied en MobaXterm

→ Usuario no es 'pi' — usar whoami para verificar

⚠ SSH deshabilitado por defecto

→ Activar en raspi-config: Interface Options > SSH

raspi-config — El panel de control del Pi

```
sudo raspi-config
```

Es como el BIOS de una laptop pero para Linux. Un menú de texto que configura todo el sistema sin necesidad de editar archivos manualmente.

`Interface Options → SSH → Enable`	Habilita el servidor SSH para control remoto. Sin esto MobaXterm no puede conectarse.
`Interface Options → VNC → Enable`	Habilita el escritorio gráfico remoto. Con MobaXterm puedes ver la GUI del Pi desde tu laptop.
`System Options → Password`	Cambia la contraseña por defecto 'raspberry'. URGENTE — cualquier atacante conoce la contraseña default.
`System Options → Wireless LAN`	Conecta el Pi a tu WiFi para soltar el cable Ethernet. Útil para modo headless definitivo.
`Localisation → Timezone`	Ajusta la zona horaria a America/Tegucigalpa para que los logs tengan hora correcta.

MobaXterm — Tu centro de control

SSH — Terminal de comandos

Session → SSH → 192.168.0.15
Usuario: raspberry | Puerto: 22

Te da una terminal de texto directa al Pi. El 90% del trabajo en ciberseguridad se hace aquí. Es como estar físicamente frente al Pi.

VNC — Escritorio gráfico remoto

Session → VNC → 192.168.0.15:5900
Contraseña: configurada con vncpasswd

Te muestra el escritorio completo del Pi en tu laptop. Útil para configuraciones visuales. Para ciberseguridad no lo necesitas tanto como SSH.

¿Por qué MobaXterm y no otros?

Todo en uno

SSH + VNC + SFTP + terminal local — un solo programa

Gratis (Home)

La versión gratuita es suficiente para todo lo que hacemos

SFTP integrado

El panel izquierdo muestra los archivos del Pi — arrastrar y soltar

Sesiones guardadas

Guarda IPs y usuarios — un doble clic y estás conectado



SECCIÓN 3: HERRAMIENTAS INSTALADAS

Qué hace cada una y por qué importa

Herramientas de ciberseguridad instaladas en el Pi

Nmap

CV: Esencial

```
sudo nmap -sV 192.168.0.0/24
```

Descubre dispositivos en la red, puertos abiertos y versiones de servicios. Es el scanner de red más usado en el mundo.

Wireshark

CV: Esencial

```
wireshark
```

Captura y analiza todo el tráfico de red en tiempo real. Ves exactamente qué datos viajan entre dispositivos. Versión gráfica.

Tcpdump

CV: Alto

```
sudo tcpdump -i eth0 -w cap.pcap
```

Captura de paquetes por terminal, más ligero que Wireshark. Ideal para dejar corriendo en background y analizar después.

Fail2ban

CV: Alto

```
sudo fail2ban-client status sshd
```

Monitorea logs de acceso y bloquea automáticamente IPs con demasiados intentos fallidos. Tu primera línea de defensa.

Nmap — Entender los escaneos

Nmap no es solo 'buscar dispositivos'. Cada tipo de escaneo tiene un propósito diferente:

Ping Scan

```
sudo nmap -sn 192.168.0.0/24
```

Solo descubre qué IPs están activas. Con sudo muestra MACs y fabricante. El más rápido.

Service Scan

```
nmap -sV 192.168.0.X
```

Detecta qué servicios corren y sus versiones. Ej: OpenSSH 8.4, Apache 2.4.51.

OS Detection

```
nmap -sV -O 192.168.0.X
```

Intenta adivinar el sistema operativo del objetivo. Requiere sudo para máxima precisión.

Full Port Scan

```
nmap -p- --min-rate 5000 IP
```

Escanea los 65535 puertos. Lento pero completo. min-rate lo acelera.

Default Scripts

```
nmap -sC -sV IP
```

Ejecuta scripts básicos de detección. Estándar en HTB/TryHackMe como primer escaneo.

Vuln Scan

```
nmap --script vuln IP
```

Busca vulnerabilidades conocidas. Más agresivo — no usar en redes ajenas.



SECCIÓN 4: VIAJE CON LA ESP32

Firmwares, errores, soluciones y MicroPython

Cronología ESP32 — Lo que pasó y por qué

1 Detección inicial

✅ OK

ls /dev/ttyUSB* → /dev/ttyUSB0
chip_id → ESP32-D0WD-V3 rev3.1, MAC: 44:1d:64:f9:69:f4

2 Intento BT Sniffer

⚠ Parcial

Flasheamos firmware BTSnifferBREDR. Resultado: SUCCESS pero firmware no grabó correctamente.

3 Error UnicodeDecodeError

❌ Error

El sniffer enviaba datos binarios (byte 0xa6). MicroPython en la flash causaba incompatibilidad.

4 Flash borrada

✅ Normal

erase_flash limpió la ESP32. Estado: invalid header 0xffffffff — flash vacía, no dañada.

5 MicroPython instalado

✅ OK

write-flash con ESP32_GENERIC-20241129-v1.24.1.bin. 1.69MB en 102 segundos. Hash verificado.

6 ESP32 en la red WiFi

✅ OK

minicom → MicroPython REPL → ifconfig() → IP: 192.168.0.22 obtenida automáticamente.

MicroPython en la ESP32 — Comandos aprendidos

MicroPython es Python 3 optimizado para microcontroladores. Corre directamente en la ESP32 sin sistema operativo.

Desde minicom (-D /dev/ttyUSB0 -b 115200) con el prompt >>>

Conectar al WiFi

```
import network
w = network.WLAN(network.STA_IF)
w.active(True)
w.connect('NombreRed', 'Contrasena')
print(w.ifconfig())  # IP, mascara, gateway, DNS
```

Escanear redes WiFi cercanas

```
import network
w = network.WLAN(network.STA_IF)
w.active(True)
# Devuelve: (SSID, BSSID, canal, RSSI, auth, hidden)
print(w.scan())
```

Ver MAC de la ESP32

```
import ubinascii
mac = w.config('mac')
print(ubinascii.hexlify(mac, ':').decode())
# Resultado: 44:1d:64:f9:69:f4
```

Conexion socket al Pi

```
import socket
s = socket.socket()
s.connect(('192.168.0.15', 22))
print('Conectado al Pi!')
s.close()
```

Errores encontrados — Qué significan y qué enseñan

`UnicodeDecodeError: 'utf-8' codec can't decode byte 0xa6`

🔍 Causa: El programa esperaba texto UTF-8 pero recibió datos binarios. La ESP32 tenía MicroPython y enviaba su prompt de arranque en binario.

💡 Lección: Siempre verifica el firmware antes de correr software que depende de él. Borrar y regrabar es la solución.

`invalid header: 0xffffffff (en minicom)`

🔍 Causa: La flash de la ESP32 estaba completamente vacía (erase_flash la había borrado). El bootloader no encontraba ningún programa.

💡 Lección: NO significa daño permanente. Es el estado normal después de borrar. La solución es simplemente grabar un firmware nuevo.

`ImportError: no module named 'uping'`

🔍 Causa: uping no es un módulo estándar de MicroPython. Algunos módulos de Python normal no existen en MicroPython por limitaciones de memoria.

💡 Lección: MicroPython tiene una librería más pequeña que Python normal. Siempre verifica disponibilidad del módulo.

`Warning: Command 'write_flash' is deprecated`

🔍 Causa: esptool v5+ cambió la sintaxis de sus comandos. El guión bajo fue reemplazado por guión medio.

💡 Lección: Las herramientas se actualizan y cambian su sintaxis. Cuando ves 'deprecated', usa la nueva versión del comando.



SECCIÓN 5: HOJA DE RUTA

Lo que sigue — Fases de aprendizaje

Hoja de ruta — 5 fases de aprendizaje

FASE 1

Cowrie Honeypot

⏳ Pendiente

- Completar instalación pendiente
- Ver ataques SSH reales en logs
- Analizar IPs atacantes en Shodan

FASE 2

ESP32 como herramienta

🔄 En progreso

- Escanear redes WiFi cercanas
- Generar tráfico para capturar
- Sensor IoT vulnerable para atacar

FASE 3

Docker + Servicios

📋 Planificado

- NGINX, API, base de datos en Pi
- DVWA para practicar web hacking
- Segmentación de red real

FASE 4

Mini SOC

📋 Planificado

- Grafana + rsyslog dashboards
- IDS con alertas en tiempo real
- Detección de amenazas activa

FASE 5

Ataque vs Defensa

📋 Planificado

- Kali VM ataca, Pi defiende
- Metasploit + máquinas vulnerables
- HTB/TryHackMe con laboratorio propio

Comandos maestros — Referencia rápida completa

Pi — Sistema

```
sudo shutdown -h now
```

```
sudo reboot
```

```
hostname -I
```

```
sudo raspi-config
```

```
sudo apt update && sudo apt upgrade -y
```

```
sudo systemctl status ssh
```

```
sudo tail -f /var/log/auth.log
```

Nmap

```
sudo nmap -sn 192.168.0.0/24
```

```
nmap -sV -O 192.168.0.X
```

```
nmap -sC -sV IP_objetivo
```

```
nmap -p- --min-rate 5000 IP
```

```
sudo tcpdump -i eth0 -w cap.pcap
```

```
sudo fail2ban-client status sshd
```

ESP32 + MicroPython

```
ls /dev/ttyUSB*
```

```
python3 -m esptool chip_id
```

```
python3 -m esptool erase_flash
```

```
minicom -D /dev/ttyUSB0 -b 115200
```

```
import network; w=network.WLAN()
```

```
print(w.ifconfig())
```



LABORATORIO EN CONSTRUCCIÓN

El objetivo no es llegar al final — es entender cada pieza del camino

SSH sin monitor ✅	VNC funcionando ✅	Nmap en red local ✅
ESP32 con MicroPython ✅	IP en red WiFi ✅	Errores comprendidos ✅

Próximo: Cowrie Honeypot → Docker → Mini SOC → Ataque vs Defensa